

FAQ

FAQ — Оптимизация графика работы отделений банка

Частые вопросы участников. Если ответа нет — спрашивайте куратора.

Про постановку задачи

В чём вообще суть? Что от нас хотят в итоге?

Две вещи. Первое — модель очереди: программа, которая по числу открытых окон и потоку клиентов считает, сколько люди будут ждать. Второе — расписание: кто из сотрудников в какие часы работает, чтобы в пик окон хватало, а в затишье не простаивали. Главная метрика — среднее время ожидания клиента. Ограничение — бюджет на зарплаты (ФОТ) и трудовое законодательство (40 ч/неделя, обед).

Нужно ли писать веб-сервис или интерфейс?

Нет. Достаточно скриптов или Jupyter-ноутбука. Веб-сервис писать не нужно — это математический проект, а не веб-разработка. Красивые графики важнее красивого интерфейса.

Насколько это «настоящая» банковская задача?

Настоящая. Планирование смен в отделениях (workforce scheduling) — реальная головная боль ритейл-банков. Данные синтетические, но модель потока и структура ограничений взяты из практики.

Можно ли решить задачу «на минимум» и всё равно получить хорошую оценку?

Да. Корректная симуляция очереди + разумное расписание по часам — это уже добротное решение. Продвинутое вещи (формула Эрланга, оптимизация по бюджету) поднимают оценку, но не являются обязательными. Лучше сделать базу хорошо, чем всё поверхностно.

Про данные

Что за файлы в датасете и с чего начать?

Начните с `client_arrivals.csv` — это визиты клиентов. И с `README.md` — там описаны все поля. Первым делом постройте график: сколько клиентов приходит в каждый час. Сразу увидите пики.

Откуда взялись данные? Они реальные?

Синтетические, сгенерированы специально для задачи. Реальные банковские данные — это тайна, их выдать нельзя. Но статистические свойства (неравномерность потока, разброс времени обслуживания) реалистичны.

Что такое `service_time_min` — это время в очереди или время обслуживания?

Только обслуживание — сколько минут клиент провёл непосредственно у окна. Время ожидания в очереди в данных НЕ записано, потому что оно зависит от расписания, которое вы и должны построить. Ожидание вы будете считать сами в симуляции.

Почему у одинаковых операций разное время обслуживания?

Потому что в жизни так. Один перевод занял 3 минуты, другой — 7. В данных это заложено: время обслуживания — случайная величина со средним и разбросом (см. `operations.csv`, поля `avg_service_min` и `std_service_min`).

В данных есть воскресенья?

Нет. Отделения по воскресеньям закрыты. Суббота — сокращённый день (9:00–16:00). Пн–пт — полный (9:00–19:00).

Можно ли использовать дополнительные данные / генерировать свои?

Генератор данных вам не выдаётся (он у кураторов). Но вы можете написать собственный, если хотите проверить решение на других сценариях — это даже плюс к оценке. Главное — итоговые результаты показывайте на выданном датасете.

Про математику

Я не знаю теорию массового обслуживания. Это проблема?

Нет. Всё нужное можно освоить за пару часов. Ключевые понятия: пуассоновский поток (клиенты приходят случайно, но с известной средней частотой), интенсивность λ (сколько в среднем клиентов в час), система М/М/с (с окон обслуживания). Хорошая новость: даже без формул можно всё посчитать симуляцией.

Что такое пуассоновский процесс простыми словами?

Модель случайных событий, которые происходят независимо друг от друга с постоянной средней частотой. Классический пример — звонки в колл-центр. Если в среднем приходит 12 клиентов в час, это не значит «ровно по одному каждые 5 минут» — иногда трое подряд, иногда пусто десять минут. Пуассон описывает именно такую случайность.

Обязательно ли использовать формулу Эрланга?

Нет. Формула Эрланга С — это аналитический способ посчитать время ожидания для М/М/с без симуляции. Это красиво и быстро, но не обязательно. Можно всё решить симуляцией (моделируя каждого клиента). Если используете и то, и другое и сравните — отлично, это сильный ход.

Что такое «симуляция очереди»? Как это программируется?

Вы моделируете время по шагам (или по событиям). Клиенты приходят по расписанию из данных, встают в очередь, обслуживаются, когда окно освобождается. Для каждого клиента вы записываете: когда пришёл, когда начал обслуживаться, сколько прождал. В простейшем виде это цикл на 50 строк. Библиотека `simpy` умеет это делать «из коробки», но можно и вручную.

Насколько сложная математика ожидается? Мы же не матшкола.

Порог низкий: пуассоновское распределение, среднее и дисперсия, немного комбинаторики для расписания. Всё это — первый курс. Глубина набирается не сложностью формул, а качеством анализа: насколько аккуратно вы оценили параметры, проверили модель, обосновали выводы.

Про реализацию

Какой язык и библиотеки использовать?

Python. Библиотеки: pandas (данные), numpy (вычисления), matplotlib или plotly (графики). Опционально: simpy (симуляция), scipy.optimize (оптимизация). Ничего экзотического не требуется.

Можно ли использовать готовые библиотеки для очередей / оптимизации?

Да, можно. Но если библиотека делает всю работу за вас (вы вызвали одну функцию и получили ответ) — обязательно покажите, что понимаете, что происходит внутри. Жюри спросит. Понимание важнее, чем факт использования библиотеки.

Насколько большой код ожидается?

Небольшой. Это не про объём кода, а про идеи. Хорошее решение — это 200–400 строк аккуратного, понятного кода, а не тысячи строк. Если пишете много — вероятно, усложняете.

А если у нас разный уровень в команде?

Нормально. Разделите роли: кто-то делает визуализацию и анализ данных (нужен только pandas), кто-то — симуляцию (нужна логика), кто-то — расписание и оптимизацию (нужна математика), кто-то — отчёт и презентацию. Каждому найдётся задача по силам.

Про оценку и защиту

Как будут оценивать?

Пять критериев: качество модели очереди (30%), качество расписания (25%), математическая глубина (20%), визуализация и презентация (15%), оригинальность (10%). Подробности — в полном описании проекта.

Что показать на защите, чтобы впечатлить жюри?

Три вещи. Наглядную визуализацию нагрузки (heatmap «час × день» заходит всегда). Работющую симуляцию, где видно, как меняется ожидание при изменении числа окон. И честный анализ: где ваше решение хорошо, а где — нет. Признание слабых мест ценится выше, чем их замалчивание.

Что чаще всего забывают сделать?

Проверить, что симуляция вообще правильная. Прежде чем строить расписание, убедитесь, что ваша модель очереди даёт разумные числа (например, сравните с формулой Эрланга на простом случае). Расписание, построенное на неправильной модели, бессмысленно.

Можно ли принести решение только части задачи?

Да. Частичное, но качественное решение оценивается выше, чем полное, но поверхностное. Лучше идеально сделать симуляцию и базовое расписание, чем накидать всё сразу и нигде не довести до ума.

Организационное

Сколько человек в команде?

3–5 человек.

Сколько времени займёт?

Проект рассчитан на две недели мастерской. Первая неделя — данные, анализ, модель очереди. Вторая — расписание, оптимизация, отчёт и презентация.

К кому обращаться с вопросами?

К куратору проекта от Совкомбанк Технологий. Технические вопросы по данным и постановке — тоже к нему.